Unit 4 Progress Check: MCQ Part C

1. Consider the following code segment.

```
int[][] table = {{9, 8, 7}, {6, 5, 4}, {3, 2, 1}};
System.out.println(table[2][1]);
```

What value is printed as a result of executing the code segment?

(A) ✓
2

(B)
3

(C)
4

(D)
6

Answer A

Correct. The value 2 is stored in table[2][1]. The first square bracket indicates the row index and the second square bracket indicates the column index. Two-dimensional array indices for both rows and columns begin at 0.

Unit 4 Progress Check: MCQ Part C

2. Consider the following code segment, which is intended to create and initialize the two-dimensional array num so that columns with an even index will contain only even integers and columns with an odd index will contain only odd integers.

```
int[][] num = /* missing code */;
```

Which of the following initializer lists could replace /* missing code */ so that the code segment will work as intended?

(A) ✓

```
{{0, 1, 2}, {4, 5, 6}, {8, 3, 6}}
```

(B)

```
{{1, 2, 3}, {3, 4, 5}, {5, 6, 7}}
```

(C)

```
{{2, 1, 4}, {5, 2, 3}, {2, 7, 1}}
```

(D)

```
{{2, 4, 6}, {1, 3, 5}, {6, 4, 2}}
```

Answer A

Correct. Two-dimensional arrays are arrays of arrays. This initializer list will create a 2D array with 3 rows and 3 columns. The first row (with index 0) will be assigned the values 0, 1, and 2. The second row (with index 1) will be assigned the values 4, 5, and 6. Finally, the third row (with index 2) will be assigned the values 8, 3, and 6. Therefore, the first column (with index 0) will contain the values 0, 4, and 8. The second column (with index 1) will contain the values 1, 5, and 3. Finally, the third column (with index 2) will contain the values 2, 6, and 6. So the first and third columns (with even indices) contain only even integers and the second column (with an odd index) contains only odd integers.

Unit 4 Progress Check: MCQ Part C

3. The following method is intended to return the total number of elements in the 2D array arr. The method does not work as intended.

```
public int totalElements(int[][] arr)
{
  return arr.length * arr.length;   // Line 3
}
```

Which of the following changes can be made so that the code segment works as intended?

(A) Changing line 3 to return (arr.length - 1) * (arr.length - 1);

(B) Changing line 3 to return arr.length + arr.length;

(C) Changing line 3 to return (arr.length - 1) * (arr[0].length - 1);

(D) Changing line 3 to arr.length * arr[0].length; ✓

Answer D

Correct. The expression arr.length evaluates to the number of rows in the array, and the expression arr.length[0] evaluates to the number of columns in the array. The method currently uses arr.length * arr.length, which incorrectly multiplies the number of rows by itself. The total number of elements in a 2D array can be calculated by multiplying the number of rows (arr.length) by the number of columns (arr[0].length).

Unit 4 Progress Check: MCQ Part C

4. Consider the following code segment.

```
int[][] mat = {{1, 2, 3, 4},
               {5, 6, 7, 8},
               {8, 7, 6, 5},
               {4, 3, 2, 1}};

for (int r = 0; r < mat.length; r++)
{
  /* missing code */
}
```

This code segment is intended to produce the following output.

```
1
5 6
8 7 6
4 3 2 1
```

Which of the following can be used to replace `/* missing code */` so that the code segment works as intended?

Unit 4 Progress Check: MCQ Part C

(A)

```
for (int c = 0; c <= r; c++)
{
  System.out.print(mat[c][r] + " ");
}
System.out.println();
```

(B) ✓

```
for (int c = 0; c <= r; c++)
{
  System.out.print(mat[r][c] + " ");
}
System.out.println();
```

(C)

```
for (int c = 1; c < r; c++)
{
  System.out.print(mat[r][c] + " ");
}
System.out.println();
```

Unit 4 Progress Check: MCQ Part C

(D)

```
for (int c = 0; c <= r; c++)
{
  System.out.print(mat[r][c] + " ");
  System.out.println();
}
```

Answer B

Correct. The outer loop will traverse from 0 to `mat.length`, which is the number of rows. For each value of the outer loop, the inner loop will traverse from 0 to `r`, the current row setting. When `r = 0`, `c` will go from 0 to 0, printing one element (`mat[0][0]`) from the first row. When `r` is 1, `c` will go from 0 to 1, printing 2 values from the second row (`mat[1][0]` and `mat[1][1]`). The traversal continues with each row printing an additional element until `r` is 3 and `c` goes from 0 to 3, printing all four values in that row. This would produce the intended output.

Unit 4 Progress Check: MCQ Part C

5. Consider the following code segment.

```
int[][] board = new int[5][4];

/* code to initialize board */

int total = 0;
for (int r = 1; r < board.length; r += 2)
{
  for (int c = 0; c < board[0].length; c++)
  {
    total += board[r][c];
  }
}
```

Which of the following best describes the value of total as a result of executing the code segment?

(A) It contains the sum of elements in even-numbered rows.

(B) It contains the sum of elements in odd-numbered rows. ✓

(C) It contains the sum of elements in even-numbered columns.

(D) It contains the sum of elements in odd-numbered columns.

Answer B

Correct. Only the rows with odd numbered indexes will be traversed because the outer loop control variable is initialized to 1 and incremented by 2. All columns are traversed by the inner loop.

Unit 4 Progress Check: MCQ Part C

6. Consider the following method, sumRows, which is intended to traverse all the rows in the two-dimensional (2D) integer array num and print the sum of all the elements in each row.

```
public static void sumRows(int[][] num)
{
  for (int[] r : num)
  {
    int sum = 0;
    for (int j = 0; j < num.length; j++)
    {
      sum += r[j];
    }
    System.out.print(sum + " ");
  }
}
```

For example, if num contains {{3, 5}, {6, 8}}, then sumRows(num) should print the following.

```
8 14
```

The method does not always work as intended.

For which of the following two-dimensional array input values does sumRows **not** work as intended?

(A)

```
{{10, -18}, {48, 17}}
```

(B)

```
{{-5, 2, 0}, {4, 11, 0}}
```

(C) ✓

```
{{4, 1, 7}, {-10, -11, -12}}
```

(D)

```
{{1, 2, 3}, {4, 5, 6}, {7, 8, 9}}
```

Answer C

Unit 4 Progress Check: MCQ Part C

Correct. For this option, the inner `for` loop will iterate two times since the two-dimensional array has two rows. However, there are three elements in each row. So the sum of only the first two elements in each row is printed. The Boolean expression in the inner `for` loop should be `j < r.length` instead of `j < num.length`.

7. The following method is intended to return `true` if all elements in row `x` of the 2D array `nums` are positive and `false` otherwise.

```
/**
 *  Precondition: x is a valid row index of nums.
 */
public static boolean allPositiveInRow(int[][]nums, int x)
{
  boolean allPositive = /* initial value */;
  for (int y = 0; y < nums[0].length; y++)
  {
    if (nums[x][y] <= 0)
    {
      allPositive = /* updated value */;
    }
  }
  return allPositive;
}
```

Which of the following replacements for `/* initial value */` and `/* updated value */` can be used so that the method works as intended?

(A) Replace `/* initial value */` with `true` and replace `/* updated value */` with `true`.

(B) Replace `/* initial value */` with `true` and replace `/* updated value */` with `false`. ✓

(C) Replace `/* initial value */` with `false` and replace `/* updated value */` with `true`.

(D) Replace `/* initial value */` with `false` and replace `/* updated value */` with `false`.

Answer B

Correct. The `boolean` variable is initialized to `true` and is only updated to be `false` if a non-positive value is found in the row.

Unit 4 Progress Check: MCQ Part C

8. Consider the following code segment.

```
int[][] mat = {{1, 2, 3, 4},
               {1, 2, 3, 4},
               {5, 6, 7, 8},
               {5, 6, 7, 8}};

int numRows = mat.length;
int numCols = mat[0].length;
int [][] changedMat = new int[numRows][numCols];
for (int row = 0; row < numRows; row++)
{
  for (int col = 0; col < numCols; col++)
  {
    changedMat[numRows – row – 1][col] = mat[row][col];
  }
}
```

Which of the following represents the contents of changedMat after executing this code segment?

Unit 4 Progress Check: MCQ Part C

(A) ✓

{{5, 6, 7, 8},

{5, 6, 7, 8},

{1, 2, 3, 4},

{1, 2, 3, 4}}

(B)

{{1, 2, 3, 4},

{5, 6, 7, 8},

{1, 2, 3, 4},

{5, 6, 7, 8}}

(C)

{{4, 3, 2, 1},

{4, 3, 2, 1},

{8, 7, 6, 5},

{8, 7, 6, 5}}

(D)

{{1, 2, 3, 4},

{1, 2, 3, 4},

{5, 6, 7, 8},

{5, 6, 7, 8}}

Answer A

Correct. The first iteration of the inner loop copies the values of the first row (mat[0][col]) into the last row (changedMat[3][col]). The second iteration copies the values of the second row (mat[1][col]) into the second-to-last

Unit 4 Progress Check: MCQ Part C

row (changedMat[2][col]). The process continues until all rows have been copied.

9. In the following code segment, `board` is a properly declared and initialized 2D array of `int` values.

```
int sum = 0;
for (int r = 0; r < board.length; r++)        // Line 2
{
  for (int c = 0; c < board[0].length; c++)   // Line 4
  {
    if (c % 2 != 0)                           // Line 6
    {
      sum += board[r][c];                     // Line 8
    }
  }
}
```

The code segment is intended to compute the sum of all elements in all rows that have odd-numbered row indices. However, the code segment is not working as intended. Which of the following changes can be made so that the code segment works as intended?

(A) Changing line 2 to `for (int r = 0; r < board.length; r += 2)`

(B) Changing line 4 to `for (int c = 1; c < board[0].length; c += 2)`

(C) Changing line 6 to `if (r % 2 != 0)` ✓

(D) Changing line 8 to `sum += board[c][r];`

Answer C

Correct. The value of the outer loop control variable needs to be used to determine if the index of a row is odd so that only the elements in the appropriate rows are accessed.

Unit 4 Progress Check: MCQ Part C

10. Consider the following method.

```
public static int findTarget(int[] nums, int target)
{
  int answer = -1;
  for (int j = nums.length - 1; j >= nums.length / 2; j--)
  {
    if (nums[j] == target)
    {
      answer = j;
    }
  }
  return answer;
}
```

The following code segment appears in another method in the same class as findTarget.

```
int[] arr = {4, 5, 8, 3, 8, 9, 8, 1};
System.out.println(findTarget(arr, 8));
```

What is printed as a result of executing the code segment?

(A) 2

(B) 4 ✓

(C) 5

(D) 6

Answer B

Correct. The loop starts at the end of the array and works its way backward through the second half of the array. Each time the array element equals the value of target, the value of answer will be updated.

Unit 4 Progress Check: MCQ Part C

11. Consider the following method.

```
public static int someSearch(int[] arr, int value)
{
  for (int j = 0; j < arr.length; j++)
  {
    if (arr[j] == value)
    {
      return j;
    }
  }
  return -1;
}
```

The following code segment appears in a method in the same class as `someSearch`.

```
int[] myNumbers = {1, 2, 5, 2, 6, 3, 3, 5, 3};
System.out.println(someSearch(myNumbers, 3));
```

What is printed as a result of executing the code segment?

(A)

-1

(B) ✓

5

(C)

6

(D)

8

Answer B

Correct. The `someSearch` method performs a linear search of `arr` from left to right. Although `3` appears multiple times in `arr`, the method returns the index of the first element in `arr` that matches value. In this case,

Unit 4 Progress Check: MCQ Part C

that occurs when `j` is equal to `5`.

12. Consider the method `linearSearch`, which takes an `ArrayList` of `Integer` elements and a target `int` value as parameters and returns the index of the first appearance of the target value in the list or `-1` if the target value does not appear in the list.

```
public static int linearSearch(ArrayList<Integer> elements, int target)
{
  for (int j = 0; j < elements.size(); j++)     // Line 3
  {
    if (elements.get(j) == target)
    {
      return j;
    }
  }
  return -1;
}
```

Which of the following describes how replacing line 3 with `for (int j = (elements.size() - 1); j >= 0; j--)` will affect the behavior of `linearSearch`?

(A) The modification has no effect: the modified method will continue to return the index of the first appearance of the target value in the list, or `-1` if the target value does not appear in the list.

(B) The modified method will return the index of the last appearance of the target value in the list, or `-1` if the target value does not appear in the list. ✓

(C) The modified method will throw an `IndexOutOfBoundsException`.

(D) The modified method will return `-1` regardless of the inputs.

Answer B

Correct. Modifying the `linearSearch` method in that way causes it to search the list from right to left instead of from left to right. As a result, when the target value does appear in the list, the index of the last appearance of the target value is returned instead of the index of the first appearance.

Unit 4 Progress Check: MCQ Part C

13. The following method is a correct implementation of the selection sort algorithm. The method correctly sorts the elements of `arr` so that they appear in order from least to greatest.

```
public static void selectionSort(int[] arr)
{
  for (int j = 0; j < arr.length - 1; j++)
  {
    int minIndex = j;
    for (int k = j + 1; k < arr.length; k++)
    {
      if (arr[k] < arr[minIndex])
      {
        minIndex = k;
      }
    }
    int temp = arr[j];
    arr[j] = arr[minIndex];
    arr[minIndex] = temp;
    /* end of outer loop */
  }
}
```

Which of the following could be the contents of `arr` after two passes of the outer loop (i.e., when `j == 1` at the point indicated by `/* end of outer loop */`)?

(A) ✓

{10, 20, 50, 40, 30}

(B)

{10, 30, 50, 40, 20}

(C)

{20, 10, 50, 40, 30}

(D)

{20, 30, 50, 40, 10}

Answer A

Unit 4 Progress Check: MCQ Part C

Correct. After two passes in selection sort, the smallest two elements must be in the first and second positions in the sorted array.

Unit 4 Progress Check: MCQ Part C

14. The following method is intended to sort an array of integers into ascending order.

```
public static void sort(int[] arr)
{
  for (int j = 0; j < arr.length - 1; j++)
  {
    int minIndex = j;
    for (int k = j + 1; k < arr.length - 1; k++)
    {
      if (arr[k] < arr[minIndex])
      {
        minIndex = k;
      }
    }
    int temp = arr[minIndex];
    arr[minIndex] = arr[j];
    arr[j] = temp;
  }
}
```

This method works as intended for some, but not all, int arrays. Which of the following arrays can be used to show that the method does not work as intended?

(A)

{1, 2, 3, 4, 5}

(B) ✓

{1, 2, 3, 5, 4}

(C)

{1, 2, 4, 3, 5}

(D)

{1, 3, 2, 4, 5}

Answer B

Correct. The last element in the array is never processed by the method since the inner loop ends at arr.length - 1; thus, it will not be sorted when completed.

Unit 4 Progress Check: MCQ Part C

15. Consider the following correct implementation of the selection sort algorithm.

```
public static void selectionSort(int[] elements)
{
  for (int j = 0; j < elements.length - 1; j++)
  {
    int minIndex = j;
    for (int k = j + 1; k < elements.length; k++)
    {
      if (elements[k] < elements[minIndex])
      {
        minIndex = k;
      }
    }
    if (j != minIndex)
    {
      int temp = elements[j];
      elements[j] = elements[minIndex];
      elements[minIndex] = temp;   // line 17
    }
  }
}
```

The following declaration and method call appear in a method in the same class as selectionSort.

```
int[] arr = {30, 40, 10, 50, 20};
selectionSort(arr);
```

How many times is the statement elements[minIndex] = temp; in line 17 of the method executed as a result of the call to selectionSort?

(A) 2

(B) 3 ✓

(C) 4

(D) 5

Answer B

Correct. The statement in line 17 executes each time a value is swapped into the correct position in the array. Three swaps would occur. For the given array, the values 30 and 10 are swapped. Then the values 40 and 20 are

swapped. Then, since 30 is already in the correct position, no swap occurs. Lastly, 50 and 40 are swapped.

16. Consider the following method.

```
public static int mystery(int n)
{
  if (n < 10)
  {
    return n;
  }
  return (n % 10) + mystery(n / 10);
}
```

Assume that the `int` variable `num` is properly declared and initialized to a value greater than 0. Which of the following best describes the value returned by the call `mystery(num)`?

(A) The value of `num`

(B) The rightmost digit of `num`

(C) The sum of the digits of `num` ✓

(D) Two times the rightmost digit of `num`

Answer C

Correct. The recursive case takes the rightmost digit (`n % 10`) and adds it to the result of recursively summing the other digits.

Unit 4 Progress Check: MCQ Part C

17. The following recursive method is intended to return `true` if `origNum` is divisible by each digit in `numsToDivideBy`, and to return `false` otherwise. For example, `isDivisible(236, 24)` should return `true` because `236` is divisible by both `2` and `4`. The method does not always work as intended.

```
public static boolean isDivisible(int origNum, int numsToDivideBy)
{
  if (numsToDivideBy == 0)
  {
    return true;
  }
  else if (origNum % (numsToDivideBy % 10) != 0)
  {
    return false;
  }
  else
  {
    return isDivisible(origNum, numsToDivideBy / 10);
  }
}
```

Which of the following method calls can be used to demonstrate that the method does **not** work as intended?

(A)

isDivisible(120, 12)

(B)

isDivisible(135, 5)

(C) ✓

isDivisible(225, 105)

(D)

isDivisible(296, 341)

Answer C

Correct. The original call `isDivisible(225, 105)` first checks whether 225 is divisible by 5 (since `105 % 10 == 5`). Both the `if (numsToDivideBy == 0)` and `else if`

Unit 4 Progress Check: MCQ Part C

`(origNum % (numsToDivideBy % 10) != 0)` conditions evaluate to `false`, so the call `isDivisible(225, 10)` is made. On that call, the `else if` statement attempts to evaluate `225 % (10 % 10)`, which is equal to `225 % 0`, and throws an `ArithmeticException`. The method has no check to avoid dividing by zero.

Unit 4 Progress Check: MCQ Part C

18. Consider the following recursive method.

```
public static int mystery(int x)
{
  if (x <= 0)
  {
    return 5;
  }
  else
  {
    return x + mystery(x - 2);
  }
}
```

What is returned as a result of the method call `mystery(5)`?

(A)

8

(B)

13

(C) ✓

14

(D)

20

Answer C

Correct. The call `mystery(5)` returns `5 + mystery(3)`. The call `mystery(3)` returns `3 + mystery(1)`. The call `mystery(1)` returns `1 + mystery(-1)`. The call `mystery(-1)` matches the base case and returns `5`. This ends the recursion and `5 + 3 + 1 + 5`, or `14`, is returned.

Unit 4 Progress Check: MCQ Part C

19. The following recursive method is intended to return the index of the first occurrence of goal in arr between index start and index end, inclusive. The index of the first occurrence of goal is the index closest to start where goal is found. If goal does not appear in the range of indices from start to end, the method returns -1. However, the method does not work as intended.

```
/** Precondition: 0 <= start <= end, end < arr.length */
public static int locateItem(int[] arr, int goal, int start, int end)
{
  if (start == end)                          // Line 4
  {
    return -1;
  }
  else if (arr[start] == goal)               // Line 8
  {
    return start;                            // Line 10
  }
  else
  {
    return locateItem(arr, goal, start + 1, end);  // Line 14
  }
}
```

Which of the following changes can be made so that the method works as intended?

(A) Changing line 4 to if (start > end) ✓

(B) Changing line 8 to else if (start == goal)

(C) Changing line 10 to return arr[start];

(D) Changing line 14 to return locateItem(arr, goal, start + 1, end + 1);

Answer A

Correct. Without this change, the method returns -1 when goal appears at index end. This change allows end to be a valid return index.

Unit 4 Progress Check: MCQ Part C

20. Consider the following method.

```
public static void find(String[] words, int s, int i)
{
  if (i != words.length)
  {
    if (i == 0 || !(words[s].equals(words[i])))
    {
      s = i;
      System.out.print(s + " ");
    }
    find(words, s, i + 1);
  }
}
```

The following code segment appears in a method in the same class as find.

```
String[] furniture = /* missing initialization */ ;
find(furniture, 0, 0);
```

Which of the following can be used to replace /* missing initialization */ so that the code segment prints 0 1 4 6 7 ?

Unit 4 Progress Check: MCQ Part C

(A)

{"Stool", "Chair", "Chair", "Chair",

"Stool", "Couch", "Couch", "Table"}

(B)

{"Table", "Couch", "Couch", "Stool",

"Chair", "Chair", "Chair", "Stool"}

(C) ✓

{"Stool", "Chair", "Chair", "Chair",

"Couch", "Couch", "Stool", "Table"}

(D)

{"Table", "Stool", "Couch", "Couch",

"Chair", "Chair", "Chair", "Stool"}

Answer C

Correct. In the first call, the first value of `s` (0) is always printed. For recursive calls, each time the next string is not equal to the previous string, `s` is updated to the new index of the current word and is then printed. Indexes of words not equal to previous words are 1, 4, 6, and 7, which is what is printed. The method makes no recursive call when `i` is beyond the last valid index of the array.

Unit 4 Progress Check: MCQ Part C

21. Consider the following recursive method.

```
public static String doSomething(String str)
{
  if (str.length() < 1)
  {
    return "";
  }
  else
  {
    return str.substring(0, 1) + doSomething(str.substring(1));
  }
}
```

Assume that `myString` is a properly declared and initialized `String` object. Which of the following best describes the result of the call `doSomething(myString)`?

(A) The method call returns a `String` containing the contents of `myString` unchanged. ✓

(B) The method call returns a `String` containing the contents of `myString` with the order of the characters reversed from their order in `myString`.

(C) The method call returns a `String` containing all but the first character of `myString`.

(D) The method call returns a `String` containing only the first and second characters of `myString`.

Answer A

Correct. When there is at least one character in `str`, the method returns the first character of `str` concatenated with the value returned by the recursive call. The parameter of the recursive call is the substring of `str` starting at the second character. As a result, the original string is reconstructed recursively.